

Modul Seri Praktikum

# Membuat Fitur Backup Transaksi Keuangan Berbasis API ExpressJS

Study Kasus

Framework7 versi 9



# Membuat Fitur Backup Transaksi Keuangan Berbasis API ExpressJS

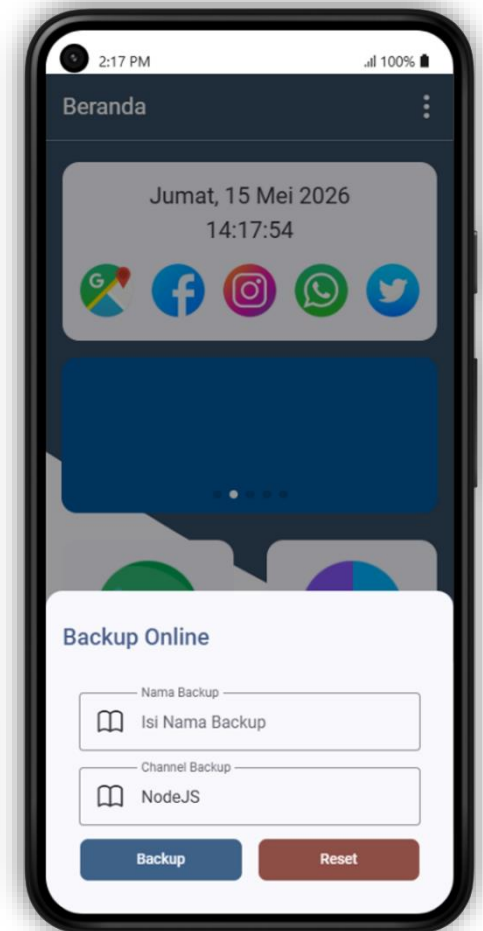
Pada praktikum kali ini kita akan **membangun sebuah fitur backup untuk transaksi keuangan yang telah dibuat dengan database offline IndexedDB**. **Fitur backup yang direncanakan menggunakan server online dengan bantuan Application Programming Interface (API) berbasis ExpressJS**. Fitur backup ini akan mengirim data yang berasal dari database **IndexedDB** yang ada di smartphone (offline) ke server online dan disimpan dalam database MySQL yang telah disiapkan. Untuk merealisasikan hal tersebut, maka kita membutuhkan API berbasis **ExpressJS** yang telah dibuat sebelumnya untuk diupload ke server online, beberapa **pengaturan dasar seperti library, koneksi database, pengaturan Middleware serta Query Database**. Dikarenakan API berbasis ExpressJS diupload ke server online, maka yang perlu diperhatikan adalah tidak semuanya support dengan project berbasis NodeJS. Opsinya antara lain:

- Menggunakan **Server Hosting** yang **support Project berbasis NodeJS**
- Menggunakan **VPS** (Virtual Private Server)
- Menggunakan **VPN** (Virtual Private Network) baik yang gratis atau berbayar.
- Menggunakan **Server Deployment** khusus **Project NodeJS**

Setelah memahami skenario yang akan dilakukan, maka langkah-langkahnya sebagai berikut

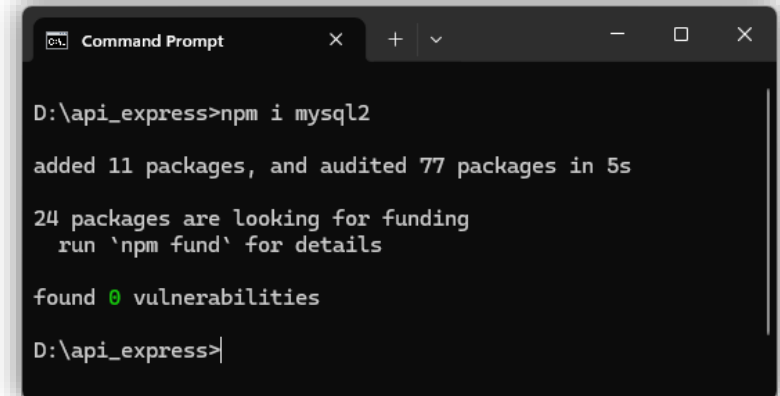
1. Sebagai langkah awal sebenarnya **kita telah memiliki pilihan untuk backup melalui API berbasis NodeJS**. Pilihan telah dibuat pada praktikum sebelumnya. Seperti yang ditunjukkan pada gambar disamping (perhatikan pada pilihan Channel Backup).
2. Perhatikan kembali bahwa **pengaturan untuk backup melalui channel API berbasis NodeJS telah tersedia di function backup\_transaksi yang ada di file beranda.f7**. Perhatikan bagian pernyataan kondisional berikut yang ada di function tersebut

```
let url = channel == "laravel" ? "https://apilaravel.crazycode.my.id/api/backup" : "https://apinodejs.crazycode.my.id/api/backup";
```



Dari syntax itu kita mengetahui bahwa url untuk API berbasis NodeJS telah tersedia. Kita tinggal menyesuaikan Project API-nya. Dalam hal ini url untuk project API berbasis NodeJS diubah menjadi <https://apinodejs.crazycode.my.id/backup> Silahkan sesuaikan url ini di function **backup\_transaksi**.

- Langkah berikutnya adalah **menginstall library mysql2** agar API berbasis NodeJS kita dapat terhubung dengan database MySQL yang telah dibuat pada praktikum sebelumnya. Silahkan ketik perintah **npm i mysql2** di **Windows Command Prompt** seperti gambar disamping
- Langkah berikutnya adalah **menginstall library cors** agar API berbasis NodeJS kita dapat menerima request dari aplikasi mobile tanpa halangan. Silahkan ketik perintah **npm i cors** di **Windows Command Prompt** seperti gambar berikut



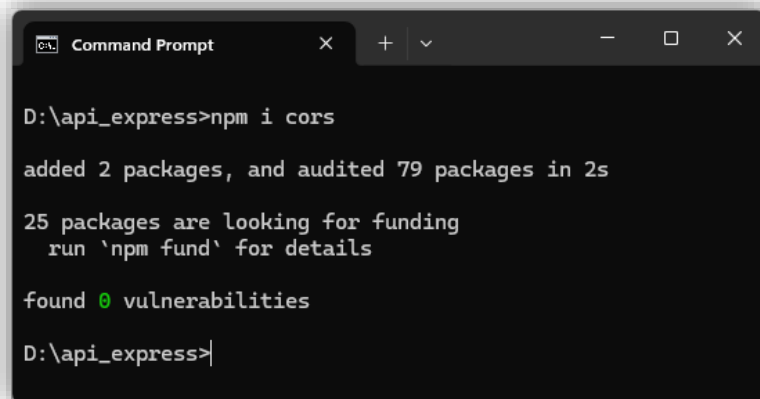
```
Command Prompt
D:\api_express>npm i mysql2

added 11 packages, and audited 77 packages in 5s

24 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities

D:\api_express>
```



```
Command Prompt
D:\api_express>npm i cors

added 2 packages, and audited 79 packages in 2s

25 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities

D:\api_express>
```

- Langkah berikutnya adalah **membuat file yang berperan seperti model (jika diibaratkan seperti Framework Laravel) yang digunakan untuk mendukung proses backup yaitu menambahkan data transaksi keuangan ke dalam database MySQL (Insert)**. File ini diberi nama **db.js** dan ditempatkan sejajar dengan file **app.js**. Didalam file db.js tersebut terdapat 3 function yaitu Function **buatKoneksi**, Function **tambahBackup** dan **tambahTransaksi**. Function **buatKoneksi** digunakan untuk menghubungkan ke database MySQL. Function **tambahBackup** digunakan **untuk menambahkan data nama backup, channel serta tanggal backupnya**, sedangkan Function **tambahTransaksi** digunakan **untuk menambah data transaksi keuangan detailnya**. Berikut merupakan contoh file **db.js** (bisa berbeda antar project silahkan sesuaikan dengan server masing-masing)

```
const mysql = require('mysql2/promise');
let sql;
const buatKoneksi = async () => {
  return await mysql.createConnection({
    host: 'localhost',
    user: '?????',
    password: '?????',
    database: '?????'
  })
}

const tambahBackup = async (id, nama, channel) => {
  const db = await buatKoneksi();
  sql = `INSERT INTO backup VALUES('${id}', '${nama}', '${channel}', NOW())`;
  try{
    await db.execute(sql);
    return "1";
  }catch(err){
    return "0";
  }
}

const tambahTransaksi = async (idx, id, waktux, nominalx, jenisx, deskripsix) => {
  const db = await buatKoneksi();
  sql = `INSERT INTO backup_transaksi VALUES('${idx}', '${id}', '${waktux}', '${nominalx}', '${jenisx}', '${deskripsix}')`;
  try{
    await db.execute(sql);
    return "1";
  }catch(err){
    return "0";
  }
}

module.exports = {buatKoneksi, tambahBackup, tambahTransaksi}
```

Dapat diperhatikan pada bagian function buatKoneksi, isian pada konfigurasi koneksi ke database masih diberi nilai awal ????. nanti jika telah diupload ke server online akan disesuaikan kembali.

6. Langkah berikutnya adalah **mengkonfigurasi file app.js** agar dapat dengan baik. Konfigurasi tersebut meliputi
- Konfigurasi terkait **CORS**. Konfigurasi ini digunakan untuk mengatur permission terkait API melalui CORS (lebih lanjut silahkan pelajari modul teori)
  - Konfigurasi terkait **db.js**. Konfigurasi ini dibutuhkan agar file app.js dapat mengakses function-function yang ada di file db.js
  - Konfigurasi terkait **Request Body API**. Konfigurasi ini dibutuhkan agar variabel-variabel yang dikirimkan melalui Body API dapat terbaca dengan baik.

Tambahkan syntax berikut setelah deklarasi variabel express.

```
const cors = require("cors");  
const db = require('./db.js');
```

Berikutnya tambahkan syntax berikut setelah deklarasi variabel port

```
app.use(cors());  
app.use(express.json());  
app.use(express.urlencoded({ extended: true }));
```

7. Langkah berikutnya adalah **membuat function yang digunakan untuk proses backup**. Function ini disebut sebagai **Middleware** dan berfungsi seperti Controller jika diibaratkan dengan Framework Laravel. **Middleware** ini ditempatkan pada **app.js**. Middleware yang dibuat bernama **backupData** dan diletakkan sejajar dengan Middleware lainnya. Berikut syntax Middleware-nya (bisa berbeda antar project silahkan sesuaikan dengan server masing-masing)

```

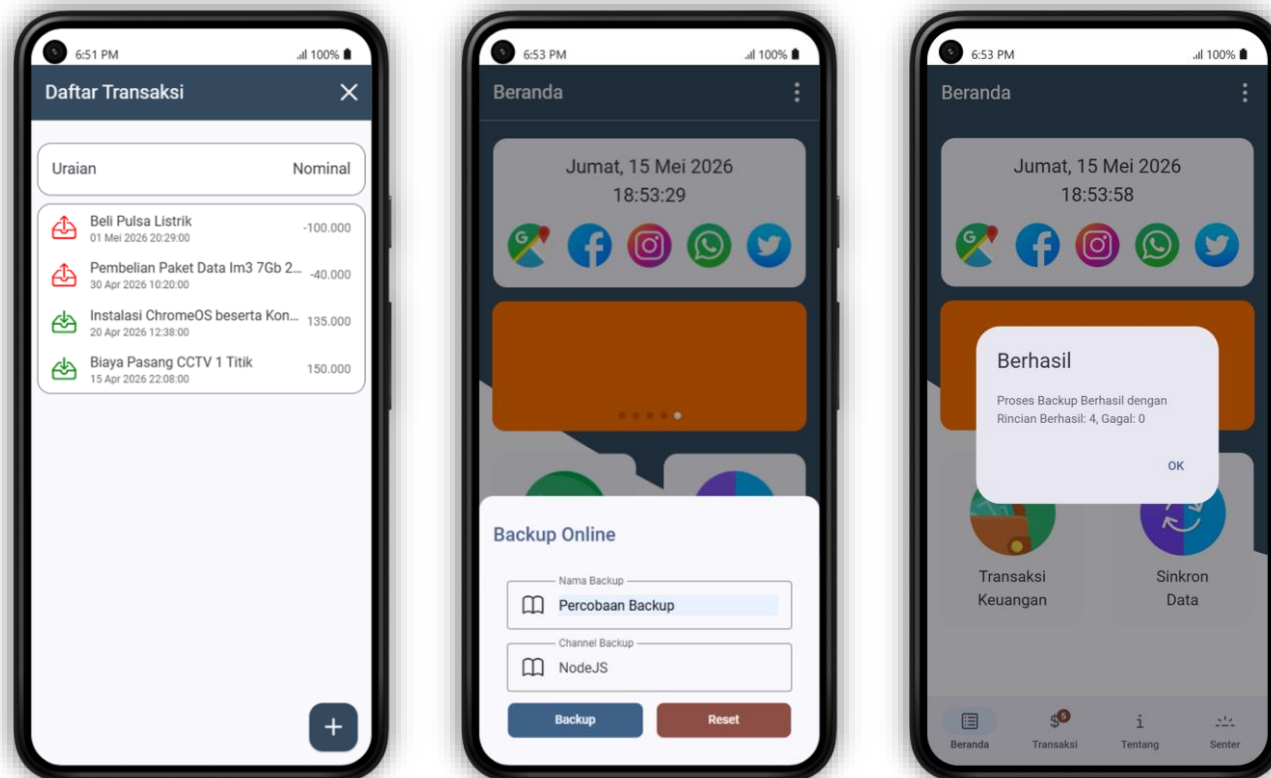
app.post("/backup", async (req, res) => {
  let pesanx, kodex;
  let nama = req.body.nama_backup;
  let dtx = atob(req.body.dtx);
  let id = Date.now();
  let arr_data = dtx.split("#");
  let proses = await db.tambahBackup(id, nama, "nodejs");
  if(proses == "1"){
    let berhasil = 0;
    let gagal = 0;
    for(k of arr_data){
      let arr_data2 = k.split("|");
      let idx = arr_data2[0];
      let deskripsix = arr_data2[1];
      let waktux = arr_data2[2];
      let nominalx = arr_data2[3];
      let jenisx = arr_data2[4];
      let proses2 = await db.tambahTransaksi(`${id}-${idx}`, id, waktux, nominalx, jenisx, deskripsix);
      proses2 == "1" ? berhasil++ : gagal++;
    }
    pesanx = {kode: "01", status: "Proses Backup Berhasil dengan Rincian ", berhasil: berhasil, gagal: gagal};
    kodex = 200;
  }else{
    pesanx = {kode: "00", status: "Proses Backup Gagal, Periksa Kembali Data Anda"};
    kodex = 500;
  }
  return res.status(kodex).json(pesanx);
})

```

8. Langkah berikutnya adalah **mengupload project API berbasis NodeJS yang telah dikonfigurasi ke server online**. Teknik menguploadnya disesuaikan dengan model server online yang digunakan apakah **server hosting, VPS, VPN** maupun **Server Deployment Project NodeJS**.



9. Langkah berikutnya adalah **mengkonfigurasi pengaturan koneksi database yang ada di function buatKoneksi yang ada di file db.js** agar project API berbasis NodeJS dapat terhubung dengan database MySQL yang telah dibuat sebelumnya. Contoh konfigurasinya seperti gambar disamping (bisa berbeda antar project silahkan sesuaikan dengan server masing-masing)
- ```
return await mysql.createConnection({
  host: 'localhost',
  user: 'mthuhsrg_usermobile',
  password: 'KXAOUh1&)k4mgJKd',
  database: 'mthuhsrg_keuangan'
})
```
10. Langkah berikutnya adalah **menguji proses backup data transaksi keuangan**. Perhatikan apakah berhasil atau tidak. Jika berhasil, maka data transaksi keuangan berhasil disimpan di database online (MySQL) seperti gambar berikut
- Skenario di Aplikasi



- Tabel Backup

|                          |                                                                                        | id                                                                                     | nama                                                                                     | channel       | waktu                                       |
|--------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|---------------|---------------------------------------------|
| <input type="checkbox"/> |  Edit |  Copy |  Delete | 1778846035561 | Percobaan Backup nodejs 2026-05-15 18:53:55 |

- Tabel Backup\_Transaksi

|                          |                                                                                        |                                                                                        | id                                                                                       | id_backup                 | tgl_jam       | nominal             | jenis    | uraian                                    |
|--------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------|---------------|---------------------|----------|-------------------------------------------|
| <input type="checkbox"/> |  Edit |  Copy |  Delete | 1778846035561-17771150112 | 1778846035561 | 2026-04-15 22:08:00 | 150000 + | Biaya Pasang CCTV 1 Titik                 |
| <input type="checkbox"/> |  Edit |  Copy |  Delete | 1778846035561-17783362748 | 1778846035561 | 2026-04-20 12:38:00 | 135000 + | Instalasi ChromeOS beserta Konfigurasinya |
| <input type="checkbox"/> |  Edit |  Copy |  Delete | 1778846035561-17783363195 | 1778846035561 | 2026-04-30 10:20:00 | 40000 -  | Pembelian Paket Data Im3 7Gb 28 Hari      |
| <input type="checkbox"/> |  Edit |  Copy |  Delete | 1778846035561-17788383372 | 1778846035561 | 2026-05-01 20:29:00 | 100000 - | Beli Pulsa Listrik                        |

Pada gambar diatas menunjukkan dari sisi aplikasi memiliki **4 data transaksi keuangan** kemudian dilakukan proses backup dan menunjukkan proses berhasil. Sebagai pembuktiannya, bisa dilihat pada database MySQL yang ada di server Online yang menunjukkan terdapat **1 data di tabel backup** dan **4 data di tabel backup\_transaksi** sesuai yang ada di aplikasi.

11. Sampai langkah ini, praktikum dianggap berhasil untuk proses backup data transaksi keuangan karena aplikasi sudah dapat terhubung dengan **API online (API NodeJS)**. Silahkan tanya pengajar jika terdapat masalah atau hal lainnya.